



Advanced Array Methods

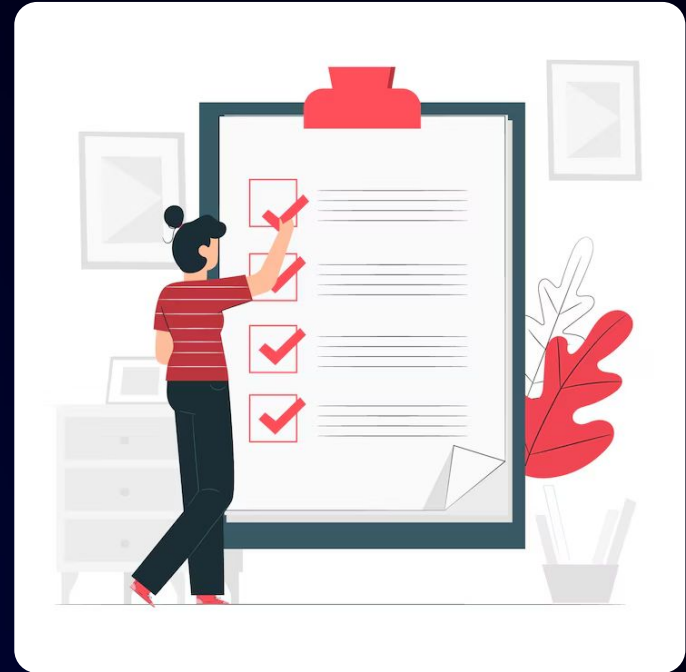
Learning Objectives

- Understand arrays and their structure
- Iterate using modern methods
- Use `map()` for transformation
- Use `filter()` for selection
- Use `reduce()` for aggregation
- Apply search methods (`find()`, `includes()`)
- Choose the right method for each scenario
- Write efficient and clean code



Prerequisites

- Basic understanding of JavaScript syntax
- Knowledge of variables and data types
- Familiarity with arrays and basic operations (create, access, update)
- Understanding of functions (including arrow functions)
- Basic knowledge of loops (for, while)
- Understanding of conditional statements (if, else)





concat **(Merging Arrays)**

Using concat Array method

- The concat method in JavaScript is used to concatenate two or more arrays into a single array
- It returns a new array that consists of the elements from the original array
- The original arrays are not modified
- Syntax - **`arr1.concat(arr2)`**



The diagram illustrates the concatenation of two arrays. It shows a light blue rounded rectangle containing the visual representation of the operation: a gray 3D box with '[1, 2]' inside, followed by a plus sign, another gray 3D box with '[5, 6]' inside, an equals sign, and a final gray 3D box with '[1, 2, 5, 6]' inside.

```
[1, 2] + [5, 6] = [1, 2, 5, 6]
```



Hands-On Demo | Using concat

Pop Quiz

Q. What will be the output of the following code in JavaScript?

```
const array1 = ['a', 'b', 'c'];
```

```
const result = array1.concat(['d', 'e'], 'f');
```

```
console.log(result);
```

A

`['a', 'b', 'c', 'd', 'e', 'f']`

B

`['a', 'b', 'c', ['d', 'e'], 'f']`

Pop Quiz

Q. What will be the output of the following code in JavaScript?

```
const array1 = ['a', 'b', 'c'];
```

```
const result = array1.concat(['d', 'e'], 'f');
```

```
console.log(result);
```

A

['a', 'b', 'c', 'd', 'e', 'f']

B

['a', 'b', 'c', ['d', 'e'], 'f']



sort **(Numbers and** **Objects)**

Using sort method

- **Purpose** - used to sort the string values in an array and customize compare function can be made to sort any given number range
- It converts the array items into string, compares them and sort the values
- The original array are not modified

Syntax -

`array.sort()`

`array.sort([compareFunction])`

```
// Functionless
sort()

// Arrow function
sort((a, b) => { /* ... */ })

// Compare function
sort(compareFn)

// Inline compare function
sort(function compareFn(a, b) { /* ... */ })
```

Using sort Array method

The compare function takes two parameters a and b and returns three possible values: 0, negative or positive.

CompareFunction(a,b) return value	Sort Order
> 0	Sort b before a
< 0	Sort a before a
== 0	Keep the original order of a and b



Hands-On Demo | Using sort

Pop Quiz

Q. What will be the output of the following code?

```
const numbers = [40, 100, 1, 5, 25, 10];
```

```
numbers.sort();
```

```
console.log(numbers);
```

A

[1, 5, 10, 25, 40, 100]

B

[1, 10, 100, 25, 40, 5]

Pop Quiz

Q. What will be the output of the following code?

```
const numbers = [40, 100, 1, 5, 25, 10];
```

```
numbers.sort();
```

```
console.log(numbers);
```

A

[1, 5, 10, 25, 40, 100]

B

[1, 10, 100, 25, 40, 5]



forEach (Iteration)

Using forEach Array method

- **Purpose** - it executes a provide function once for each array element
- This method is not executed for empty elements
- **Syntax** -

`array.forEach(callbackFunction(currentValue, index, array))`

or

`arr.forEach(() => {})`

Note - forEach does not return new array

```
// with arrow function and callback

const students = ['John', 'Sara', 'Jack'];

students.forEach(element => {
  console.log(element);
});
```




Hands-On Demo | Using forEach

Pop Quiz

Q. Which of the following correctly uses the forEach method to print each element of an array?

A

```
array.forEach((element) =>  
  console.log(element));
```

B

```
array.forEach((element) =>  
  return element);
```

Pop Quiz

Q. Which of the following correctly uses the forEach method to print each element of an array?

A

```
array.forEach((element) =>  
  console.log(element));
```

B

```
array.forEach((element) =>  
  return element);
```



map (Transform Data)

Using Map Array method

- **Purpose** - It create a new array populated with the results of calling a provided function on every element in the calling array
- This method is not executed for empty elements

- **Syntax** -

`array.map(callbackFunction(currentValue, index, array))`

Or

`array.map((currentValue, index, array) => {})`

Note - Map Array method return
new array

```
const myArray = [1,2,3,4];

const myArrayTimesTwo = myArray.map((value, index, array) => {
  return value * 2;
});

console.log(myArray); // [1,2,3,4];
console.log(myArrayTimesTwo); // [2,4,6,8];
```



Hands-On Demo | Using map

Pop Quiz

Q. How does the map method handle an empty array?

A

It returns undefined

B

It returns an empty array.

Pop Quiz

Q. How does the map method handle an empty array?

A

It returns undefined

B

It returns an empty array.



filter (Select Data)

Using filter Array method

- **Purpose** - it creates a new array with all elements that pass the test implemented by the provided function
- As the name suggest, it filter the data present in an array

- **Syntax** -

```
array.filter(function(item, index, array){})
```

Or

```
array.filter((item, index, array) => {})
```

```
Shapes = [▲, ●, ▲]  
Shapes.filter(shape => shape == ▲ )  
[▲, ▲]
```



Hands-On Demo | Using filter

Pop Quiz

Q. What will be the output of the following code?

```
const numbers = [1, 2, 3, 4, 5];
```

```
const evenNumbers = numbers.filter(num => num  
% 2 === 0);
```

```
console.log(evenNumbers);
```

A

[1, 3, 5]

B

[2, 4]

Pop Quiz

Q. What will be the output of the following code?

```
const numbers = [1, 2, 3, 4, 5];
```

```
const evenNumbers = numbers.filter(num => num  
% 2 === 0);
```

```
console.log(evenNumbers);
```

A

[1, 3, 5]

B

[2, 4]



Take a 5-minute break!

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes



reduce & reduceRight (Core Logic Builder)

Using reduce Array method

- **Purpose** -it reduce the array values to a single value.
- The reduce method runs a reducer function on each array item and returns a single output result,
- The input array can have numbers, strings or an object.
- It is not valid for empty array and does not change the original array

Using reduce Array method

- **Syntax -**

```
array.reduce(function(accumulator, currentValue, currentIndex, array), initialValue)
```

Or

```
array.reduce((accumulator, currentValue, currentIndex, array) => {}, initialValue)
```

[1, 2, 5, 6]



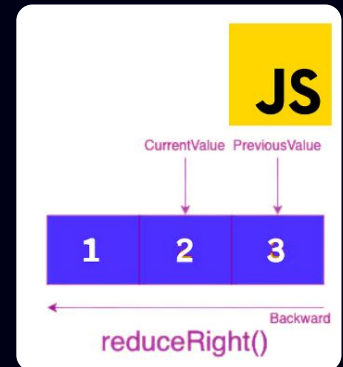
14

Using reduceRight Array method

- **Purpose** - It executes a reducer function on each element of the array (from right to left), resulting in a single output value
- **Syntax** - `array.reduceRight(callbackFunction(accumulator, currentValue, index, array), initialValue)`

Or

```
array.reduceRight((accumulator, currentValue, currentIndex, array) => {}, initialValue)
```





Hands-On Demo | `reduce` & `reduceRight`

Pop Quiz

Q. What will be the output of the following code?

```
const words = ['I', 'love', 'JavaScript'];
```

```
const sentence = words.reduce((acc, word) =>  
acc + ' ' + word);
```

```
console.log(sentence);
```

A

I love JavaScript

B

undefined

Pop Quiz

Q. What will be the output of the following code?

```
const words = ['I', 'love', 'JavaScript'];
```

```
const sentence = words.reduce((acc, word) =>  
acc + ' ' + word);
```

```
console.log(sentence);
```

A

I love JavaScript

B

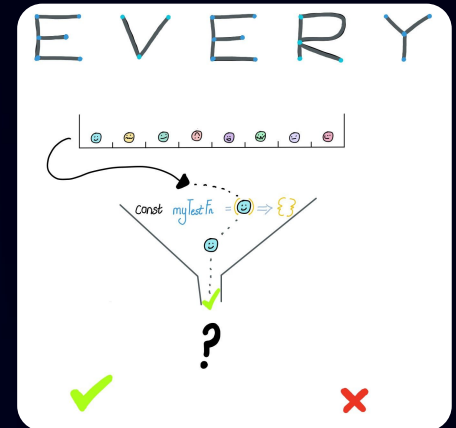
undefined



every & some

Using every Array method

- **Purpose** - Tests whether all elements in the array pass the test implemented by the provided function
- Return a boolean value
- **Syntax** - `array.every(callbackFunction(currentValue, index, array))`



Using some Array method

- **Purpose** - Tests whether at least one element in the array passes the test implemented by the provided function.
- Return a boolean value
- **Syntax** - `array.some(callbackFunction(currentValue, index, array))`

```
Shapes = [● ▲ ■ ▲ ▲]
```

```
Shapes.some(shape => shape == ▲)
```

true



Hands-On Demo | Using every & some

Pop Quiz

Q. Which of the following correctly describes the behavior of the `every()` method when called on an empty array?

A

It returns false immediately

B

It return true

Pop Quiz

Q. Which of the following correctly describes the behavior of the `every()` method when called on an empty array?

A

It returns false immediately

B

It return true



find & findIndex

Using find Array method

- **Purpose** - It is used to return the first element in an array that matches a specified condition, or undefined if no match is found.
- **Syntax** - `array.find((element, index, array) => { // condition });`

```
const numbers = [5, 12, 8, 130, 44];  
  
const found = numbers.find(element => element > 10);  
  
console.log(found); // Output: 12
```

Using indexOf Array method

- **Purpose** - It returns the first index at which a given element can be found in the array or -1 if it is not present
- **Syntax** - `array.indexOf(searchElement)` OR `array.indexOf(searchElement, fromIndexof)`

```
let array = ['a', 'b', 'c', 'd']
```

```
console.log(array.indexOf('c'))
```

```
console.log(array.indexOf('e'))
```

```
2
```

```
-1
```



Hands-On Demo | Using `find` & `findIndex`

Pop Quiz

Q. What does the indexOf() method return if the specified element is not found in the array?

A

0

B

-1

Pop Quiz

Q. What does the indexOf() method return if the specified element is not found in the array?

A

0

B

-1



**includes, indexOf,
lastIndexOf**

Using includes Array method

- **Purpose** -it determines whether an array includes a certain value among its entries, returning true or false as appropriate.
- **Syntax** - `array.includes(valueToFind, fromIndex)`

```
[1,2,3].includes(3); // true
```

```
[1,2,3].includes(4); // false
```

Using indexOf Array method

- **Purpose** - It returns the first index at which a given element can be found in the array or -1 if it is not present
- **Syntax** - `array.indexOf(searchElement)` OR `array.indexOf(searchElement, fromIndexof)`

```
let array = ['a', 'b', 'c', 'd']
```

```
console.log(array.indexOf('c'))
```

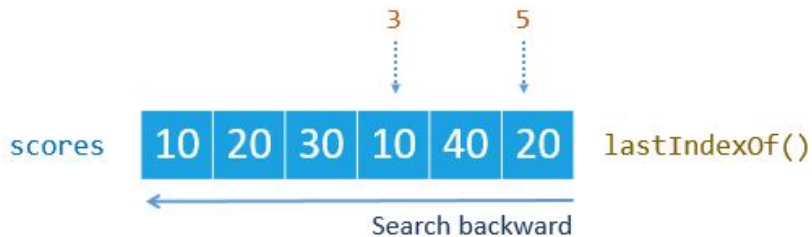
```
console.log(array.indexOf('e'))
```

```
2
```

```
-1
```

Using lastIndexOf Array method

- **Purpose** - It returns the last index at which a given element can be found in the array, or -1 if it is not present. Searches the array backward.
- **Syntax** - `array.lastIndexOf(searchElement)` OR `array.lastIndexOf(searchElement, fromIndexof)`





Hands-On Demo | using includes, indexOf, lastIndexOf

Pop Quiz

Q. What does the indexOf() method return if the specified element is not found in the array?

A

0

B

-1

Pop Quiz

Q. What does the indexOf() method return if the specified element is not found in the array?

A

0

B

-1



Practical Exercise

Advanced Array Methods

Summary

In this lecture, we have covered:

- `concat()` → merge arrays
- `sort()` → sort values (numbers/objects with compare)
- `forEach()` → iterate (no return)
- `map()` → transform data
- `filter()` → select data
- `reduce()` → combine into one value
- `every()` / `some()` → check conditions
- `find()` / `findIndex()` → search element/index
- `includes()` / `indexOf()` → check existence/position
- Practice with real examples

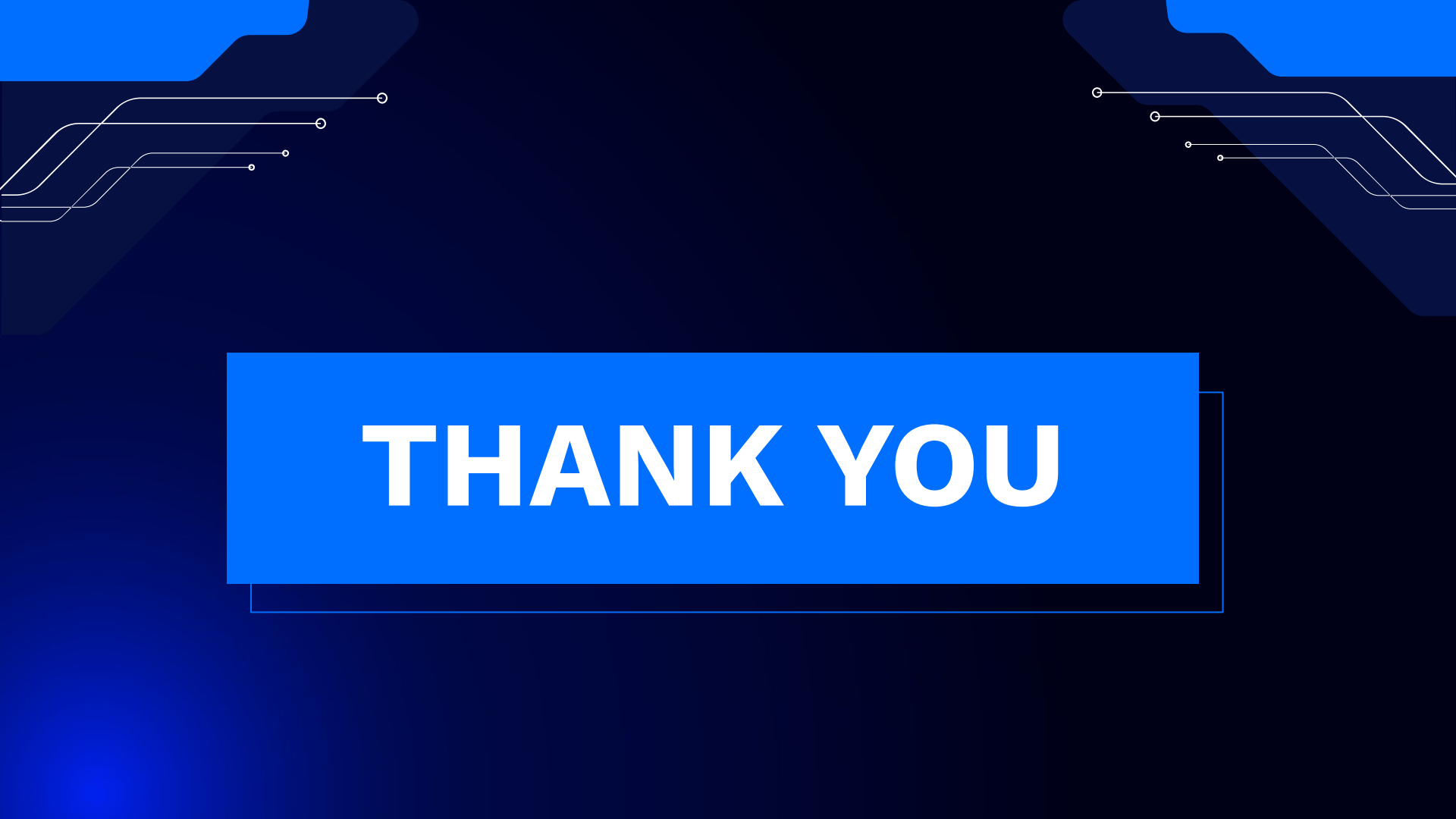




DOUBT RESOLUTION

Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.



THANK YOU